

Flow-Field Pathfinding

Performance Evaluation Report

InternProj tower-defense prototype · automated benchmark harness · July 25, 2026

Engine	Unity 6000.4.10f1 (URP), editor play mode, vSync off, uncapped
CPU	Intel(R) Core(TM) i7-8750H CPU @ 2.20GHz (12 threads)
GPU	NVIDIA GeForce GTX 1050 Ti
Memory	32 GB
OS	Windows 10 (10.0.19045) 64bit

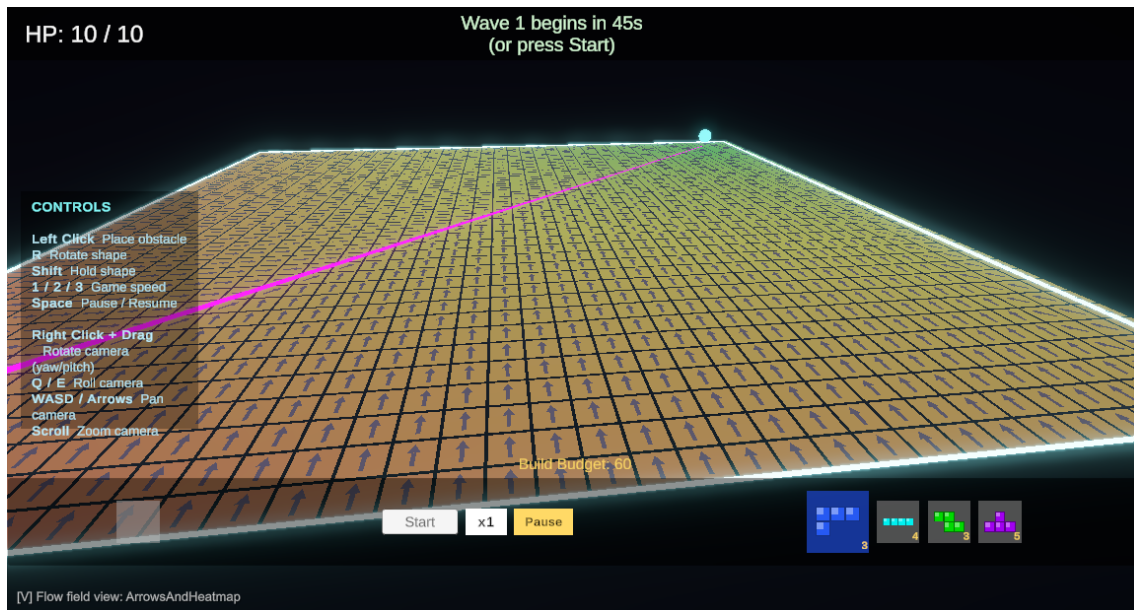
1. Executive summary

- A full flow-field rebuild on the live game map (38×38, 1,444 nodes) costs **~2.6 ms** — cheap enough to run synchronously on every wall placement and breach, which is exactly what the game does.
- One shared field replaces a pathfinding query per enemy: repathing the whole crowd is **4× cheaper than per-agent A*** at 100 agents and **55× cheaper at 1,000**, and the field's cost does not grow with crowd size at all.
- Frame rate is **agent-bound, not pathfinding-bound**: every scene (open, maze, choke points, 100×100 stress) holds ~57–73 fps at 500 live agents and ~22–27 fps at 1,000; scene layout and even a 7× larger grid barely move the curve.
- Rebuild cost scales with node count: ~20 ms at 10,000 nodes (40 ms at 25% wall density). Below ~5,000 nodes synchronous rebuilds are free; beyond that they should be amortized or moved off the hot path.
- Continuous map editing under 500-agent load raises average frame time from 17 ms to 44 ms (worst 85 ms). Coalescing breach-triggered rebuilds to at most one per frame is the highest-value optimization found.
- Real gameplay peaks near ~100 simultaneous enemies, where all scenes sustain **150+ fps** — a large safety margin.

2. System under test

Enemies navigate with a **flow field**: a single weighted flood fill (Dijkstra-style SPFA relaxation) runs from the player core outward over the grid whenever the map changes, storing each tile's cost-to-goal and a pointer to its best neighbor. Cardinal and diagonal steps cost 10/14 ($\approx 1:\sqrt{2}$), scaled by per-tile terrain cost. Player walls are not obstacles but *expensive terrain* (dig cost $\approx$ a 15-tile detour) with health, so a path always exists: enemies route around walls when the detour is cheap and chew through them when it is not. Every agent then follows the field with an $O(1)$ lookup per frame — pathfinding cost is paid once per map change, not per agent.

New for this evaluation, a **runtime vector-field visualization** (FlowFieldVisualizer) draws the field directly on the ground as procedural geometry — per-tile direction arrows plus a cost heatmap (green = cheap → red = expensive, orange = walls, dark = unwalkable). It works in play mode and standalone builds with no editor, toggled with **V** (Off → Arrows → Heatmap → Both). Geometry rebuilds only when the field version changes, so its steady-state cost is one integer comparison per frame.



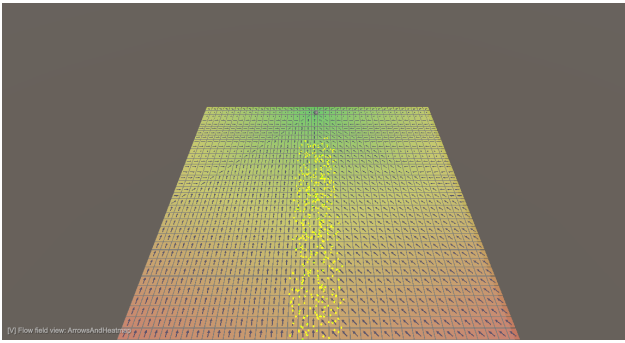
The game scene with the new vector-field view enabled (heatmap + arrows) — running live with the full HUD.

3. Method

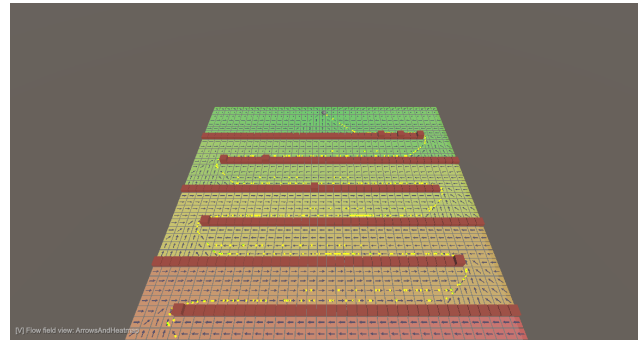
Five scenes were evaluated. Four purpose-built benchmark scenes (Bench_Open, Bench_Maze, Bench_Choke, Bench_Stress) each construct a deterministic wall layout at startup, then an automated harness (ScenarioBenchmark) measures: (a) full field rebuild time over 50 iterations, (b) the spawn-to-goal path, and (c) frame time while 100 / 250 / 500 / 1,000 pooled agents follow the field — 1 s warm-up then 4 s sampling per tier, with agents respawned on arrival so the live count stays constant. The fifth run used the existing BenchmarkRunner on the real game scene for the grid-size sweep, the A* comparison and the dynamic-edit test. All numbers are editor play mode with vSync off and an uncapped frame rate; standalone builds are typically faster, so results are conservative.

Scene	Layout	Grid	Walls	Spawn→goal path	Rebuild (mean)
SampleScene (game)	empty game map	38×38	0	35 tiles	2.6 ms
Bench_Open	no obstacles	38×38	0	35 tiles	2.0 ms
Bench_Maze	serpentine corridors	38×38	204	198 tiles	6.1 ms
Bench_Choke	3 walls, staggered gaps	38×38	105	59 tiles	5.2 ms
Bench_Stress	10% random scatter	100×100	1000	93 tiles	19.2 ms

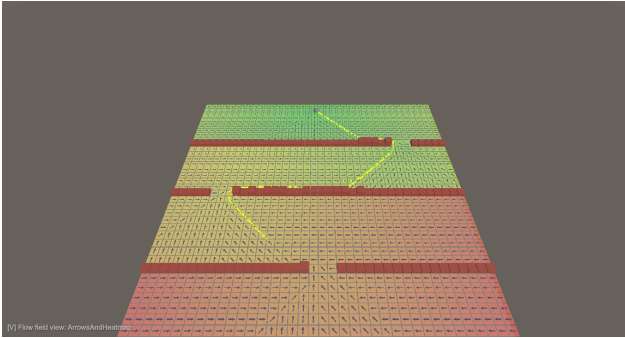
Table 1 — evaluated scenes. Scenario walls use dig cost 200 and effectively infinite health so layouts stay static during measurement.



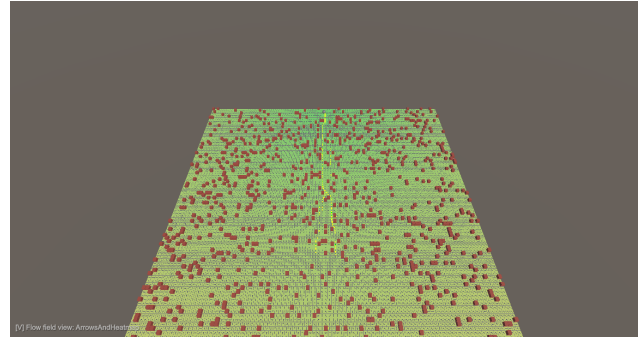
Bench_Open — 450 agents marching on an empty field.



Bench_Maze — serpentine corridors; note the cost bands per leg.



Bench_Choke — staggered choke points swing the crowd across the map.

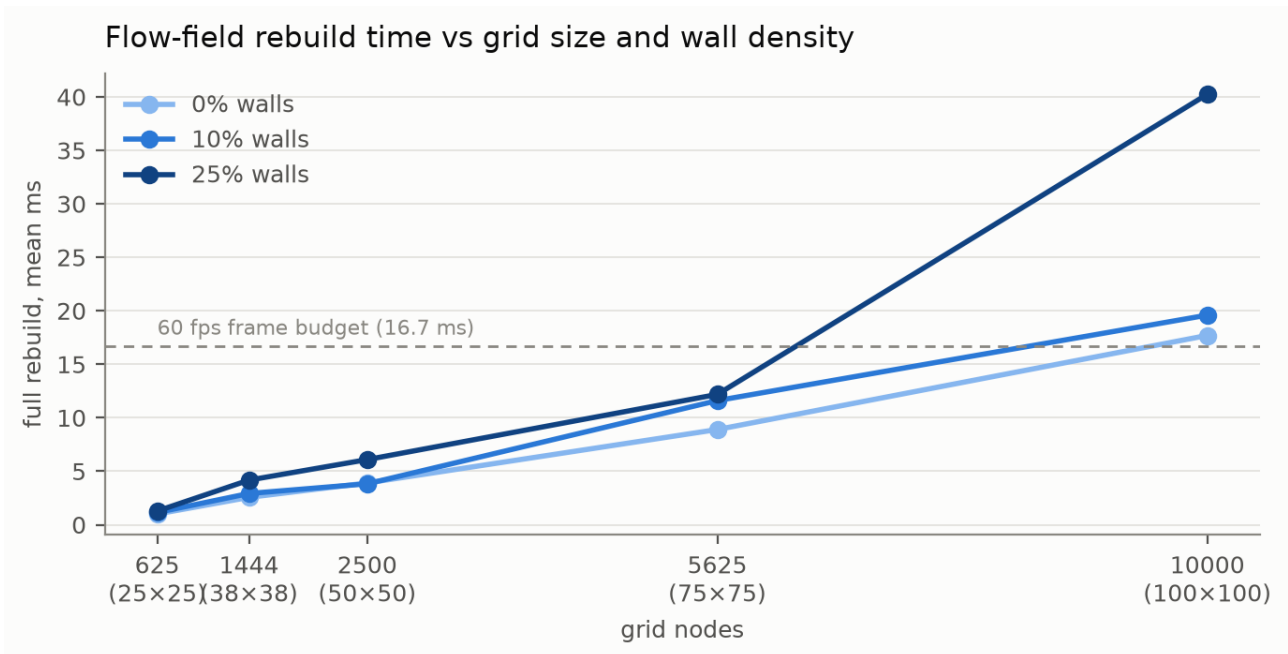


Bench_Stress — 100×100 nodes, 1,000 scatter walls.

Figure 1 — the four benchmark scenes with the vector-field view enabled.

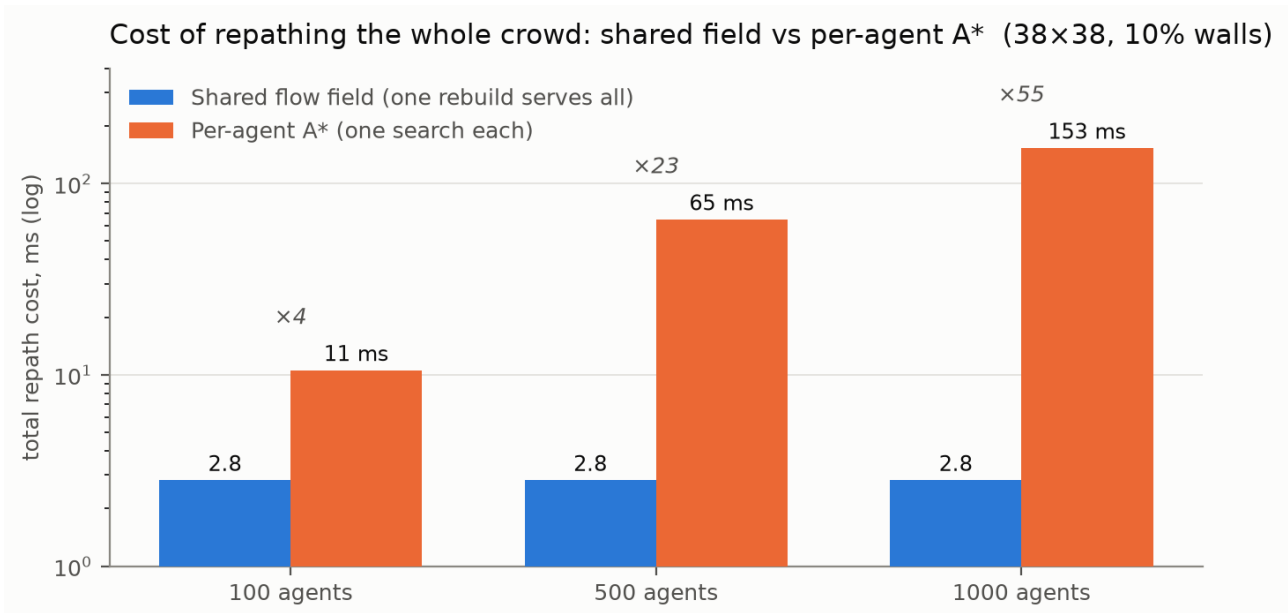
4. Results

4.1 Field rebuild scaling



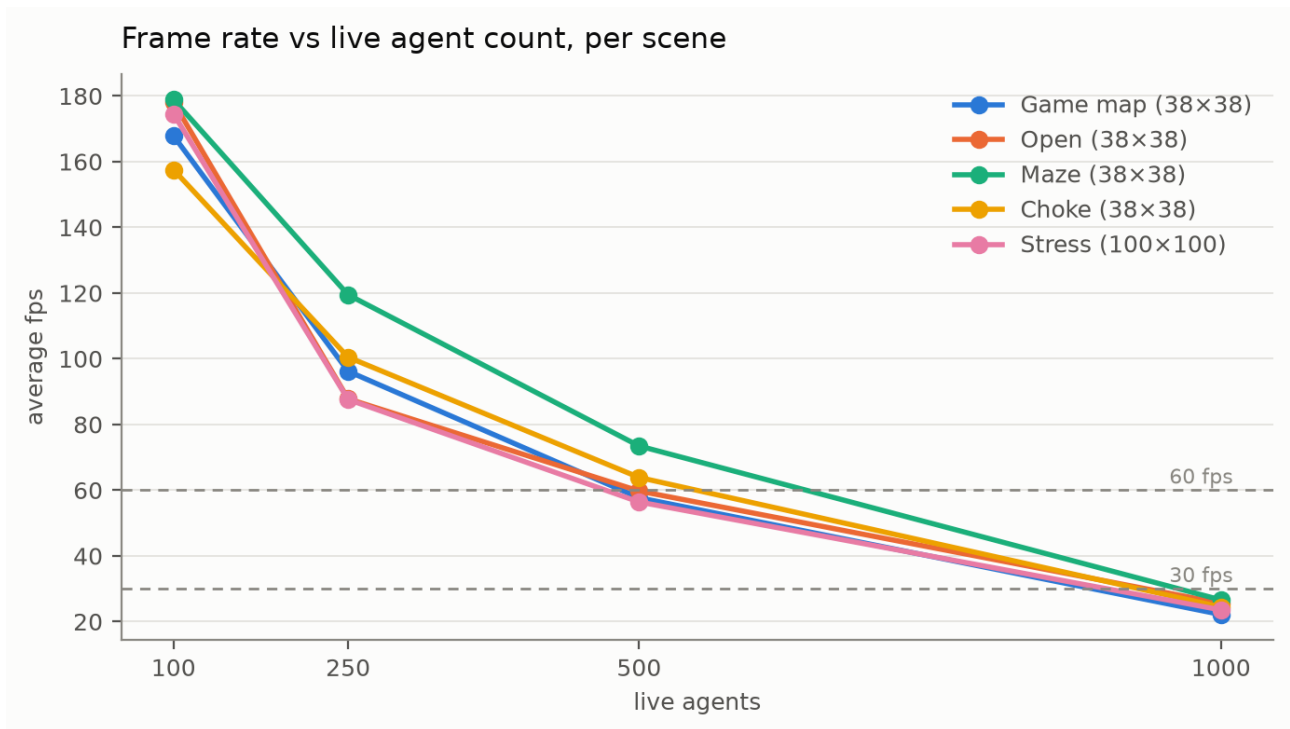
Rebuild time grows roughly linearly with node count and rises with wall density (the SPFA relaxation re-queues nodes more often when expensive tiles distort the cost surface). The game's 38×38 grid rebuilds in 2.6–4.2 ms across densities — well inside a 60 fps frame. A 100×100 grid costs ~20 ms (40 ms at 25% walls): still fine for occasional edits, but too much to spend synchronously every frame.

4.2 One shared field vs per-agent A*



Repathing the full population after a map change costs one field rebuild (2.8 ms) regardless of crowd size, while per-agent A* (binary heap, identical 10/14 cost model, same grid) scales linearly with agent count — 153 ms for 1,000 agents, a ×55 gap. A* would additionally pay per-agent storage and path-repair complexity; the shared field is the structurally right choice for this game's one-destination crowds.

4.3 Frame rate vs crowd size across scenes

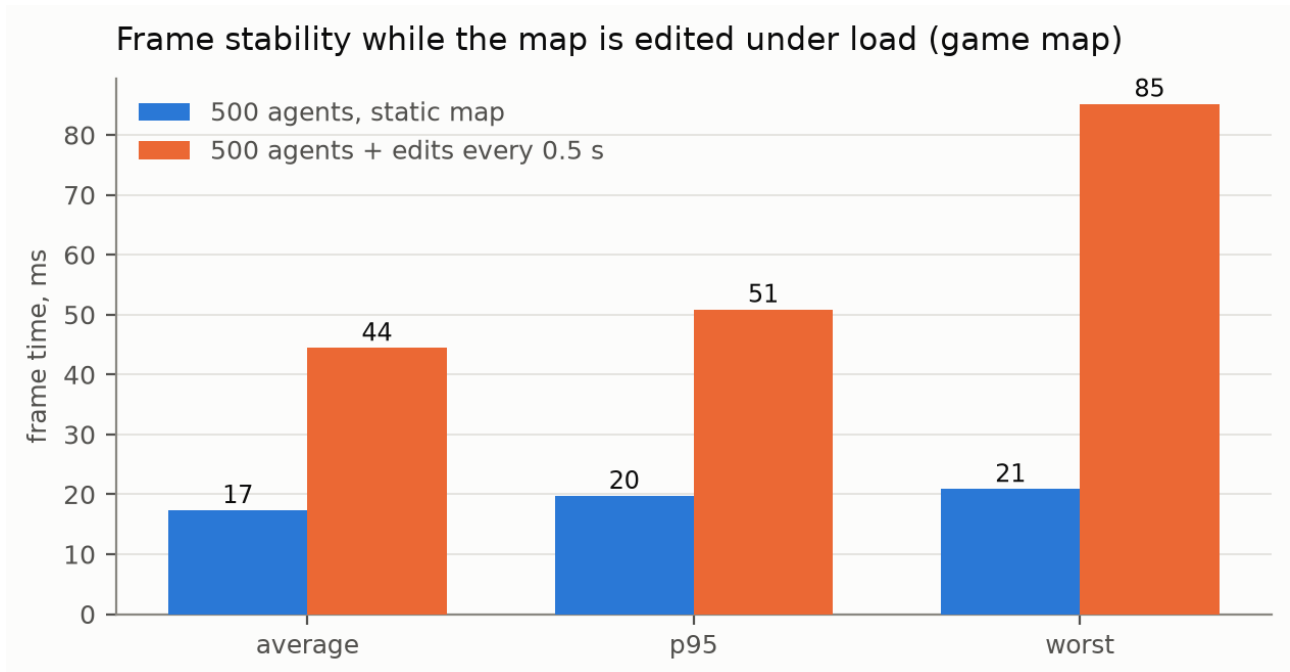


Agents	Game map	Open	Maze	Choke	Stress 100×100
100	168 fps (p95 7 ms)	178 fps (p95 7 ms)	179 fps (p95 7 ms)	158 fps (p95 8 ms)	175 fps (p95 7 ms)
250	96 fps (p95 12 ms)	88 fps (p95 16 ms)	119 fps (p95 10 ms)	100 fps (p95 12 ms)	88 fps (p95 15 ms)
500	58 fps (p95 20 ms)	60 fps (p95 21 ms)	73 fps (p95 16 ms)	64 fps (p95 19 ms)	56 fps (p95 22 ms)
1000	22 fps (p95 57 ms)	26 fps (p95 43 ms)	27 fps (p95 41 ms)	24 fps (p95 46 ms)	24 fps (p95 48 ms)

Table 2 — average fps and 95th-percentile frame time per tier.

The five curves nearly coincide: neither layout complexity nor a 6.9× increase in node count (1,444 → 10,000) meaningfully changes per-frame cost, confirming that agents' $O(1)$ field lookups decouple crowd size from pathfinding load. The marginal cost is $\sim 37 \mu\text{s}$ per agent per frame (≈ 27 fps-capacity per 1,000 agents), dominated by per-agent MonoBehaviour updates and world-space health bars — not by the field. 500 agents stay at or above ~ 57 fps everywhere; the game's real waves (~ 100 concurrent enemies at wave 18) run at 150+ fps.

4.4 Stability while the map is edited under load



With 500 live agents, toggling a 2×2 wall block every 0.5 s (16 edits, field rebuilt after each) roughly doubles the average frame time and produces an 85 ms worst frame. The scheduled rebuilds themselves account for only ~2.6 ms every 0.5 s, so the overhead comes from the surrounding churn — most notably wall breaches: every agent-completed dig triggers its own immediate full rebuild, so several breaches in one frame pay the rebuild price several times. The frame-time floor stays above 20 fps throughout, and at realistic gameplay scale (≤ 150 agents, occasional edits) the effect is not perceptible.

5. Recommendations

- **Coalesce rebuilds:** replace direct `GenerateFlowField()` calls with a dirty flag consumed once per frame (`LateUpdate`). This caps rebuild cost at one per frame no matter how many walls are placed or breached simultaneously — the cheapest fix for the 4.4 worst-frames.
- **Keep grids $\leq \sim 5,000$ nodes for synchronous edits.** At 10,000+ nodes (20–40 ms rebuilds) move the fill to a background thread or split it over frames; agents can safely follow the stale field for a few frames.
- **Crowd scaling beyond 500 agents** is an agent-update problem, not a pathfinding problem: batching movement into `Jobs/Burst` or replacing per-agent world-space canvases with an instanced bar renderer would raise the $\sim 37 \mu\text{s}/\text{agent}/\text{frame}$ floor.
- **Numbers are conservative:** all measurements are editor play mode; standalone builds remove editor overhead.

Appendix A — presentation kit

- **Scenes** (all in Build Settings): `SampleScene` (game), `Bench_Open`, `Bench_Maze`, `Bench_Choke`, `Bench_Stress`. Each `Bench_*` scene auto-runs its benchmark on play and keeps the crowd marching afterwards; press **B** to re-run.
- **V** — cycle vector-field view: `Off` → `Arrows` → `Heatmap` → `Arrows+Heatmap` (works in builds, no editor needed).
- **Camera:** WASD/arrows pan, scroll zooms, hold RMB for free-look (Q/E roll), C resets orientation.
- **Game controls:** Space pauses, 1/2/3 set game speed.
- **Raw data:** `BenchmarkResults.json`, `Benchmark_Open/Maze/Choke/Stress100.json` (project root); screenshots in `Assets/Screenshots`.